

피드백 반영 보완

1-1

지난번 정리 때부터 본문에 @Transcational로 오타로 적히는 게 반복되는 것 같아서, 혹시 몰라 얘기드려요. 실제 코드는 @Transactional로 정확하게 적으셨고 본문 타이핑할 때만 헛갈리시는 패턴 같아요. (정확한 스펠링은 transactional) 면접에서 말로 답하거나 문서에 적을 일 있을 때 한 번씩 떠올려보시면 됩니다.

네 이부분은 명심하고 면접과 문서 적을 때 주의 하겠습니다!

1-2

제 쿼리에 WHERE id = # {id}처럼 #과 {id} 사이 공백이 보입니다. 실제 코드도 이렇게 되어 있으면 MyBatis가 파라미터를 인식 못 하지 않던가요? DELETE API가 실제로 성공했는지 한 번 더 확인해주세요. 위 공백이 실제 코드에 있다면 삭제는 실패해야 정상이거든요. 캡처도 같이 보내주시면 같이 봐드릴게요.

이부분은 멘토님이 말씀하신대로 공백으로 인해서 인식을 못해서 실패가 났습니다.

공백 제거 후에 삭제 기능 수정했습니다.

잘 확인해 보겠습니다!

1-3

Controller 캡션의 "Entity 구성 로직"은 표현이 어색해요. 그 부분은 Entity가 아니라 **Controller의 조회 분기 로직**입니다.

```

@GetMapping
public ResponseEntity<List<StudentinfoResponse>> get(@RequestParam(required = false) String studentName){
    if(studentName != null){
        return ResponseEntity.ok(studentinfoService.findByStudent(studentName));
    }
    return ResponseEntity.ok(studentinfoService.findAll());
}

```

StudentinfoController 조회 분기 로직

이 부분 확인했습니다!

2. 선택보완

```

C:\Users\jdong>curl http://localhost:8080/api/studentinfo?studentName=
[]
C:\Users\jdong>curl http://localhost:8080/api/studentinfo?studentName=""
[]

```

적용 전

멘토님께서 말씀해주신?studntName= 처럼 빈 문자열이 들어오면 빈문자열 출력 되는걸 확인해서

빈 문자열도 거르게 보완했습니다.

```

@GetMapping
public ResponseEntity<List<StudentinfoResponse>> get(@RequestParam(required = false) String studentName){
    if(studentName != null && !studentName.isBlank()){
        return ResponseEntity.ok(studentinfoService.findByStudent(studentName));
    }
    return ResponseEntity.ok(studentinfoService.findAll());
}

```

적용 후

```

C:\Users\jdong>curl http://localhost:8080/api/studentinfo?studentName=""
[{"createdAt":"2026-05-10T18:18:27","id":3,"student_address":"서울시 서대문구","student_ban":"영재반","student_name":"정동훈"}, {"createdAt":"2026-05-10T02:56:39","id":2,"student_address":"서울시 영등포구","student_ban":"영재반","student_name":"김진솔"}]
C:\Users\jdong>curl http://localhost:8080/api/studentinfo?studentName=
[{"createdAt":"2026-05-10T18:18:27","id":3,"student_address":"서울시 서대문구","student_ban":"영재반","student_name":"정동훈"}, {"createdAt":"2026-05-10T02:56:39","id":2,"student_address":"서울시 영등포구","student_ban":"영재반","student_name":"김진솔"}]
C:\Users\jdong>

```

3. 4-2 정리에서 정정할 부분

3-1

생각보다 오타가 좀 많이 나는거 같아요(@Componert → @Component,YMAL → YAML)
등등

내 이부분은 @Transactional 도 그렇고 오타를 자주 낸 거 같습니다. 주의하겠습니다!

말씀해주신 부분의 오타들은 수정했습니다!

정확하게 사용하겠습니다!

```
</bean>
```

어노테이션 기반 설정은 자바 코드 위에 @Component, @Service, @Autowired 등의
어노테이션을 붙여 프레임워크에 지시를 내리는 방식입니다.

환경설정의 진화: Properties →YAML

Properties 바시요

3-2

pom이 최적 버전을 관리" → 정확히는 **BOM**(Bill of Materials)이 버전 정합성을 관리.
Starter는 의존성 묶음, BOM은 버전 관리. 면접에서 자주 묻는 부분이라 정확히 짚고 가
셔야 해요.

이부분은 멘토님이 말씀해주신 내용에 따라 BOM은 버전 관리를 해주고 Stater는 의존성
묶음이라고 이해하고 좀더 상세히 정리해봤습니다.

서버를 구축하려면 톰캣이나 JSON 변환기, 스프링 MVC 패턴과 같은 많은 라이브러리가
필요한데 과거에는 직접 적어줘야 했었는데 의존성 묶음인 Starter의 등장으로 직접 적지
않게 되었습니다.

BOM 은 최적의 정보가 적혀 있는 명세서이며 최적의 버전을 주입해주는 역할을 합니다.

3-3

패키지 구조 설명에 borad 오타도 있어요. board가 맞습니다. 실제 코드의 패키지명도
한 번 확인해보세요.

패키지 구조 만들기

@SpringBootApplication이 붙은 메인 클래스의 패키지(com.dsa.boardapi) 하위만 컴포넌트 스캔됩니다. 메인 클래스를 최상위에 두고 그아래에 board controller 같은 하위 패키지를 두는 게 정석, 빈을 못 찾는 예러가 나면 가장 먼저 패키지 위치부터 확인하세요.

네 이부분도 오타 확인했고 정리하는 내용에서 borad → board로 수정했습니다!

신경쓰겠습니다.

또 한 말씀해주신 패키지명도 그렇게 구성되어 있는지도 확인해봤는데

패키지 구성은 board로 되어있는 것을 확인했습니다. 오타에 더 신경 쓰고 주의하겠습니다.

